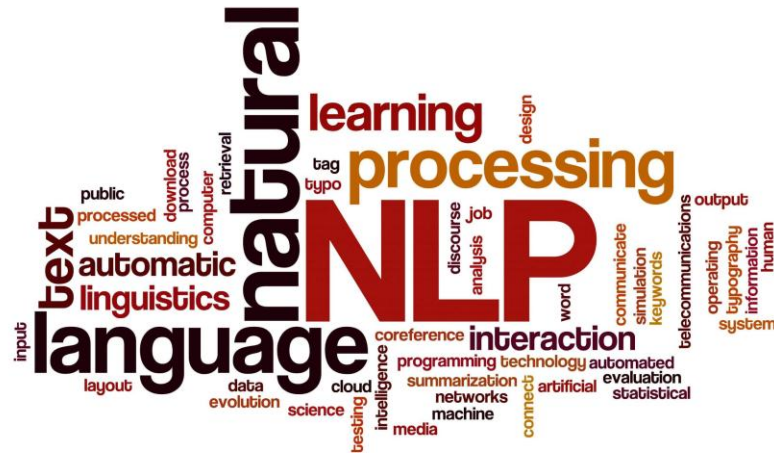


Introduction to Natural Language Processing



Pr. Hasna EL HAJI

Academic Year: 2025-2026

Introduction to Natural Language Processing

Lectures Outline

- Lecture 1: Foundations of Natural Language Processing
- Lecture 2: Word vectors & similarity
- Lecture 3: From Words to word sequences: N-grams
- Lecture 4: word2vec
- Lecture 5: Topic modeling - LSA and pLSA
- Lecture 6: RNN

Introduction to Natural Language Processing

Today's Lecture Outline

- **Representing Words in Vectors**
- **Representing Documents in Vectors**
- **Representation Techniques**
 - ❑ **One-Hot encoding** Representation
 - ❑ **Hand-Crafted Feature-Based** Representation
 - ❑ **TF-IDF Feature-Based** Representation

Framework

We assume that:

- A **token** is the basic unit of discrete data, defined to be an item from a vocabulary indexed by $1, \dots, V$.
- A **document** is a sequence of N tokens denoted by $\mathbf{d} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_N)$, where $\mathbf{w}_N$ is the N -th token in the sequence.
- A **corpus** is a collection of M documents denoted by $D = (\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_M)$.

Example: Wikipedia, All the articles from the NYT in 2025...

Token

With regard to our end task, a token can be:

- A word
- A sub-word: e.g. cardiomyopathy ["cardio", "myo", "pathy"]
- A character
- A sequence of characters (sometimes a word, sometimes several words, sometimes a sub-word...)

cardio	→	relates to heart
myo	→	muscle
Pathy	→	disease

Document

A Document can be:

- A Sentence
- A Paragraph
- A sequence of characters

Text Segmentation

Definition: Text Segmentation is the process of splitting raw text into **units of interest**.

Two levels of segmentation (usually) required :

- Split raw text into **modeling units** (ex: sentence, paragraph, 1000 characters, ...)
- Split **modeling units** into sequence of **basic units** (referred as tokens) (e.g: words, word-pieces, characters, ...)

Representing Words With Vectors

Representing Words With Vectors

Objective

Given a vocabulary $w_1, \dots, w_V$ and a corpus D , our goal is to associate each word with a representation?

What do we want from this representation?

- Identify a word
- Capture the similarities between words (based on morphology, syntax, semantics,...)
- Help us solve downstream tasks

NB: Vector-based representations of text are called embeddings

Representing Words With Vectors

One-Hot Encoding

Traditional way to represent words as atomic symbols with a unique integer associated with each word:

{1=movie, 2=hotel, 3=apple, 4=movies, 5=art}

Representation of words as One-Hot vectors:

movie = [1, 0, 0, 0, 0]

hotel = [0, 1, 0, 0, 0]

....

art = [0, 0, 0, 0, 1]

Representing Words With Vectors

One-Hot Encoding - limitations

- Implicit assumption: One-hot vectors form an orthonormal basis:
 - **Orthogonal** (mutually perpendicular)
 - **Normalized** (unit length)

- Two vectors are **orthogonal** if their dot product is 0.

Example:

$$\begin{aligned} &\text{movie} \cdot \text{art} \\ &= [1,0,0,0,0] \cdot [0,0,0,0,1] \\ &= (1 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 1) \\ &= 0 \end{aligned}$$

So, all different one-hot vectors are orthogonal.

- A vector is **normalized** if its length (L2 norm) equals 1.

For “movie”:

$$\begin{aligned} \|[1,0,0,0,0]\| &= \sqrt{1^2 + 0^2 + 0^2 + 0^2 + 0^2} \\ &= 1 \end{aligned}$$

So, each one-hot vector has norm 1.

Representing Words With Vectors

One-Hot Encoding - limitations

- Not very informative

Weird to consider “movie” and “movies” as independent entities or to consider all words equidistant: $\| \text{movie} - \text{art} \| = \| \text{movie} - \text{apple} \|^2$

- Polysemy

Should the word “mouse” referring to a computer device have the same vector representation as the animal “mouse”?

Representing Words With Vectors

Hand-Crafted Feature Representation

Example of potential features:

- **Morphology:** prefix, suffix, stem...
- **Grammar:** part of speech, gender, number,...
- **Shape:** capitalization, digit, hyphen

Those features can be defined **based on relations to other words**

- Synonyms of...
- Hypernyms of...
- Antonyms of...

We present one popular hand-crafted semantically based representation of words ⇒ **the WordNet**

Representing Words With Vectors

WordNet

Definition: a word sense is a discrete representation of one aspect of the meaning of a word.

WordNet is a large lexical database of word senses for English.

Representing Words With Vectors

WordNet

- Word types are grouped into (cognitive) synonym sets: **synsets**
Example: {Earth,earth,world,globe}
- **Polysemous** words: assigned to **different synsets**
Example: bank
- Contains **glosses** for synsets:
Example: the 3rd planet from the sun; the planet we live on
- **Noun/verb synsets**: organized in **hierarchy**, capturing IS-A relation
Example: apple IS-A fruit

Representing Words With Vectors

WordNet

X is a **hyponym** of Y if **X is an instance of Y**:

Example: cat is a hyponym of animal

X is a **hypernym** of Y if **Y is an instance of X**:

Example: animal is a hypernym of cat

X and Y are **co-hyponyms** if they have the **same hypernym**:

Example: cat and dog are co-hyponyms

Representing Words With Vectors

WordNet

X is a **meronym** of Y if **X is a part of Y**:

Example: wheel is a meronym of car

X is a **holonym** of Y if **Y is a part of X**:

Example: car is a holonym of wheel

Representing Words With Vectors

WordNet

Synset Similarity:
$$\text{Sim}(s_1, s_2) = \frac{1}{\text{length}(s_1, s_2) + 1}$$

- **Idea:** The shorter the **hypernym/hyponym** path from one synset to another, the higher is the similarity.
- **+1** avoids division by zero when the synsets are identical

Word Similarity:
$$\text{Sim}(w_1, w_2) = \max \text{Sim}(s_1, s_2)$$

Example: $\text{sim}(\text{bank}, \text{money}) = ?$

Representing Words With Vectors

WordNet

Example: $\text{sim}(\textit{bank}, \textit{money}) = ?$

Word 1: **bank** $\rightarrow$ synsets: [financial institution, river edge]

Word 2: **money** $\rightarrow$ synsets: [financial institution]

Synset similarity:

bank(financial) & money $\rightarrow 0.9$

bank(river) & money $\rightarrow 0.1$

Max strategy:

$$\text{sim}(\textit{bank}, \textit{money}) = \max(0.9, 0.1) = 0.9$$

Representing Words With Vectors

Hand-Crafted Representations - limitations

- Requires a lot of **human annotations**
 - **Subjectivity** of the annotators
 - **Does not adapt** to new words (languages are not stationary!)
- **It does not scale** easily to new languages, new concepts, new words...

Representing Words With Vectors

How to Infer “Good” Representations with Data?

- Distributional Hypothesis:
“You shall know a word by the company it keeps”, J. R. Firth (1957).
- Idea: model the context of a word to build its vectorial representation.

Representing Words With Vectors

Distributional word representation in a nutshell

- Define what is **the context** of a word
- Count how many times each target word occurs in this context
- Build vectors out of a function of these context occurrence counts:

$$x_w = f(w, \text{Context}(w))$$

Representing Words With Vectors

How to define “the context” of a word?

It can be defined as:

- The surrounding words (left and right words).
- All the remaining words of the sentence/the paragraph.
- All the words after preprocessing and filtering-out some words.

Representing Words With Vectors

How to Model the Context to get

$$x_w = f(w, \text{Context}(w))$$

Approach 1: Count-Based

1. Measure frequency of words in the context for each word in the vocabulary
2. Define vector representations based on those frequencies

Approach 2: Prediction Based

We train a model to predict words from their context

Representing Words With Vectors

Co-Occurrence Matrix

Counting the Occurrence of the words in the context of the word “dog”:

	leash	walk	run	owner	pet	barked
dog	3	5	2	5	3	2
cat	0	3	3	2	3	0
lion	0	3	2	0	1	0
light	0	0	0	0	0	0
bark	1	0	0	2	1	0
car	0	0	1	3	0	0

Representing Words With Vectors

Define vector representation based on the Co-Occurrence

	leash	walk	run	owner	pet	barked	the
dog	3	5	2	5	3	2	8
lion	0	3	2	0	1	0	6
light	0	0	0	0	0	0	5
bark	1	0	0	2	1	0	0
car	0	0	1	3	0	0	3

Naïve Approach: Take the row of the co-occurrence matrix

Representing Words With Vectors

Define vector representation based on the Co-Occurrence

	leash	walk	run	owner	pet	barked	the
dog	3	5	2	5	3	2	8
lion	0	3	2	0	1	0	6
light	0	0	0	0	0	0	5
bark	1	0	0	2	1	0	0
car	0	0	1	3	0	0	3

Limits:

- Representations depend on the size of the corpus
- Frequent words impact a lot the representations
- Representations are very sensitive to change in very infrequent words

Representing Words With Vectors

Solution: Pointwise Mutual Information (PMI)

Idea: Instead of absolute co-occurrence statistics, use probability (relative) of co-occurrences:

$$PMI(w_1, w_2) = \log \frac{P(w_1, w_2)}{P(w_1) P(w_2)}$$

Intuition: The more dependent dog and cat, the closer $P(\text{dog}, \text{cat})$ is from $P(\text{dog})P(\text{cat})$, the smaller is the PMI.

Representing Words With Vectors

Pointwise Mutual Information (PMI)

	leash	walk	run	owner	pet	barked	the
dog	2.75	2.24	3.16	2.24	2.75	3.16	1.77
lion	0	2.75	3.16	0	3.85	0	2.06
car	0	0	3.85	2.75	0	0	2.75

Word embedding vectors are the row of the PMI matrix

- We usually take the Positive PMI (assigned to 0 when negative) + Smooth unobserved pairs (Laplace smoothing: add 1)
- PMI does not depend on size of the corpus (the PMI is **normalized**)
- Much less sensitive to change in frequent words (**log**)

Representing Words With Vectors

Pointwise Mutual Information (PMI)

Limits:

- Very large matrix! Very large word vectors.
- Hard to use large vectors in practice (i.e. 1M word vocabulary)
- Cannot compare word vectors estimated on 2 different corpora unless they have exactly the same vocabulary!
- Solution: Build vectors with predefined size based on the PMI matrix
→ Dimensionality Reduction Techniques (we will cover in next chapters)

Representing Documents With Vectors

Representing Documents With Vectors

Similarly to what we saw for word-level representation we can represent **documents into vectors**.

1. Using word vectors
2. Count-Based Representations
3. Generative Probabilistic Graphical Model
4. Using language models

Representing Documents With Vectors

- Vectors allow us to **compute similarity, classify, cluster, or retrieve documents**.
- There are many ways to represent documents as vectors, such as Bag of Words (BoW), Doc2Vec, and others; in this lecture, we will focus on learning TF-IDF.

Representing Documents With Vectors

Term Frequency (TF)

- TF measures how often a word appears in a document. A higher frequency suggests greater importance.
- If a term appears frequently in a document, it is likely relevant to the document's content.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Representing Documents With Vectors

Term Frequency (TF)

Example:

- Document:
‘AI is transforming healthcare. AI helps doctors.’
- "AI" appears 2 times
- Total words = 7
- $TF(AI) = \frac{2}{7}$

So, TF tells us: *Is this word important in this specific document?*

Representing Documents With Vectors

Term Frequency (TF)

- Limits: High sensitivity to frequent words OR to very infrequent words

How to improve it?

- A word that is in all documents of the corpus (e.g. “the”) is not informative at all for the document representation, still it impacts the document vector.
- A word that appears in only 1 document is likely to be very informative of the document.

Solution:

- Add weight to the count using Inverse Document Frequency (IDF)

Representing Documents With Vectors

Inverse Document Frequency (IDF)

- Weighting the importance of each term with the document frequency
- IDF Reduces the weight of common words across multiple documents while increasing the weight of rare words. If a term appears in fewer documents, it is more likely to be meaningful and specific.

$$\text{IDF}(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

NB: The log smooths the impact of words that are in only a few documents

Representing Documents With Vectors

Inverse Document Frequency (IDF)

Example:

Suppose we have 100 documents:

- "the" appears in 100 documents: $IDF(the) = \log(100/100) = \log(1) = 0$
- "mammography" appears in 2 documents: $IDF(mammography) = \log(100/2)$
- $\Rightarrow$ Rare words get higher importance.

Representing Documents With Vectors

TF-IDF

- TF-IDF is computed by multiplying the Term Frequency (TF) of words by their Inverse Document Frequency (IDF).
- When applied to a collection of documents, this produces a matrix where each row is a document and each column is a word.
- The matrix is usually **sparse** (many zeros) because not all words appear in every document.
- This matrix is what we use for **document similarity, classification, and clustering**.
- Can be fed into **machine learning models**.